

Realize LIVE



Realize
LIVE



Teamcenter Extension / Customization

Rahul Thakur

Senior Technical Product Manager

Siemens Digital Industries Software

Agenda

Why customize Teamcenter

What can be customized

Introducing Scripted Business Logic

Customization of Active Workspace

Agenda



Why Customize

Tailoring the Digital Thread to meet unique business needs

Page 5 | Unrestricted | © Siemens 2025 | Siemens Digital Industries Software

SIEMENS



What can be Customized

Adapting Teamcenter for Your Business

Page 7 | Unrestricted | © Siemens 2025 | Siemens Digital Industries Software

SIEMENS



Introducing – Scripted Business Logic

A Modern Approach to Teamcenter Customization

Page 9 | Unrestricted | © Siemens 2025 | Siemens Digital Industries Software

SIEMENS



Customization of Active Workspace

Ways to customize Active Workspace

Page 19 | Unrestricted | © Siemens 2025 | Siemens Digital Industries Software

SIEMENS



Siemens Web Framework Fundamentals

Configuration points

Page 25 | Unrestricted | © Siemens 2025 | Siemens Digital Industries Software

SIEMENS



UI Builder

Configuration Over Code

Page 28 | Unrestricted | © Siemens 2025 | Siemens Digital Industries Software

SIEMENS

Extend

Why Customize

Tailoring the Digital Thread to meet unique business needs

Why do companies customize Teamcenter

Tailoring the Digital Thread to meet unique business needs

- **Process Alignment** - Mirrors real-world engineering, manufacturing, or regulatory processes
- **Business Rules & Logic** - Enforce business-specific rules
- **Integration with Enterprise Systems** - Connect with ERP, CAD, Enterprise systems
- **Enhanced User Experience** - Workflows and UIs to simplify usability for different roles to improve adoption
- **Reporting & Compliance** - Generate specific reports, tracking changes
- **Competitive Advantage** - Tailor Teamcenter to support proprietary workflows, automation, or analytics, companies gain speed, agility, and better decision-making capabilities

Extend

What can be Customized

Adapting Teamcenter for Your Business

What can be customized in Teamcenter

From Configuration to Code – Adapting Teamcenter for Your Business

- **User interface** - Add pages, arranging properties for clients - Active Workspace and Rich Client
- **Behavior** - Workflow handlers, Runtime properties, Pre-or post-actions on operations
- **Data Model** - Create new objects and properties using BMIDE
- **Localization** - Change object and property names using user's language
- **Command-line Utilities** - Write command-line utilities to access Teamcenter information

● Client-Side Customization ● Server-Side Customization

Extend

Introducing – Scripted Business Logic

A Modern Approach to Teamcenter Customization

Customization of Teamcenter

Scripted business logic - A Modern Approach to Teamcenter Customization



Simplify the server-side extensions using Scripted Business Logic in C# for Teamcenter deployments

- Scripted Business Logic using C# helps to simplify the server-side extensions while protecting existing customization investment
- Writing Teamcenter extensions in C# helps Rapid application development and also provides a modern development experience
- Extensions are interpreted rather than compiled facilitating faster iteration on business logic extensions

Customization of Teamcenter

Scripted business logic - Simplified development

.NET framework with C#

No downtime or compilation

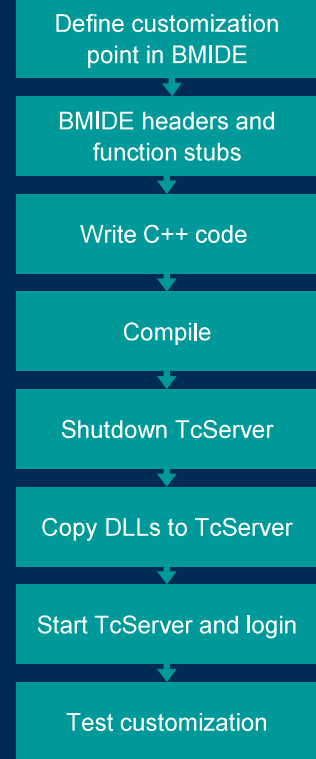
Integrated debugging and testing

Complete ITK and Metamodel coverage

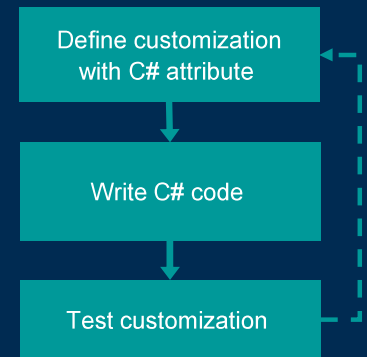
Works with existing customizations

Windows and Linux

ITK Customization (C++)

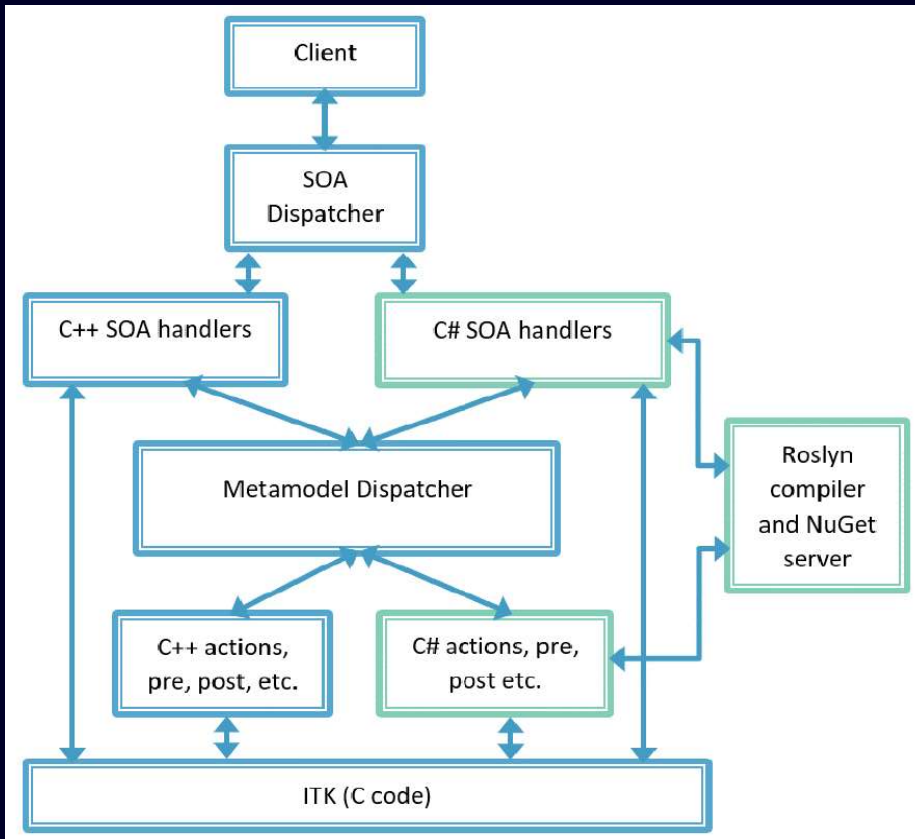


Scripted Customization



Customization of Teamcenter

Scripted business logic - Simplified development

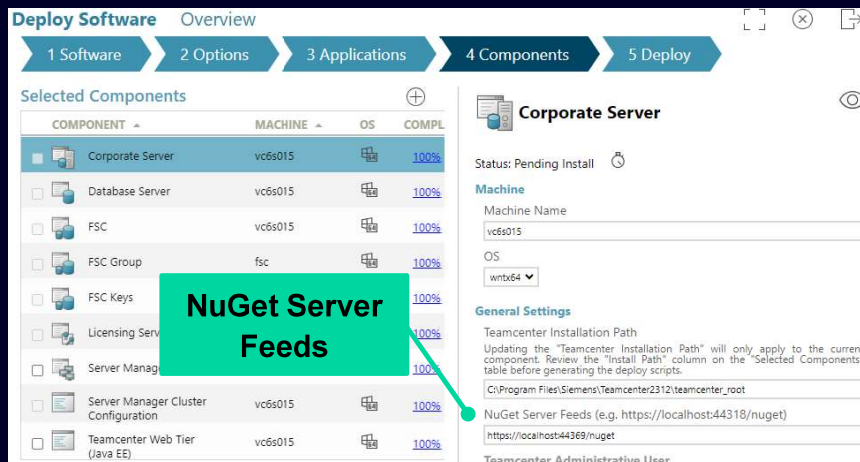
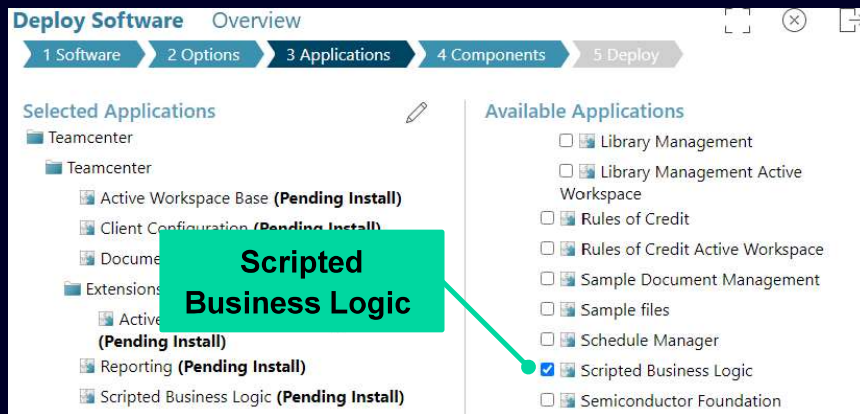


.NET customization allows software developers to write codeful extension of Teamcenter Server in C#

- Business logic created in C# will have full access to ITK and Metamodel operations
- TcServer will compile and load the business logic each time a customization source file changes without having to restart the server
- Business logic created in C# will fully interoperate with customizations created in C++
- Provides an integrated debugging and testing experience

Customization of Teamcenter

Scripted Business Logic – Deployment guidelines

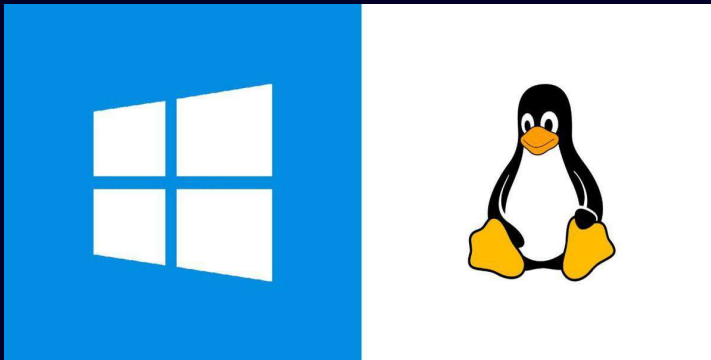
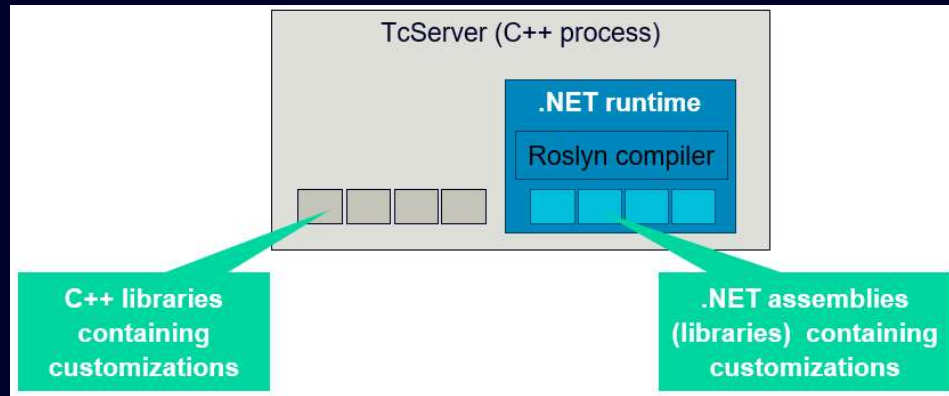


Deploy the Scripted Business Logic application using the Deployment Center

- Select the Scripted Business Logic application to deploy the artifacts needed to customize TcServer using C#
- Specify the NuGet Server Feeds address for the Deployment Center to push the out-of-box NuGet package to the server
- The NuGet server contains all the packages for either development or production environment. The publicly available packages can be hosted on your own server or can be consumed from public NuGet server

Customization of Teamcenter

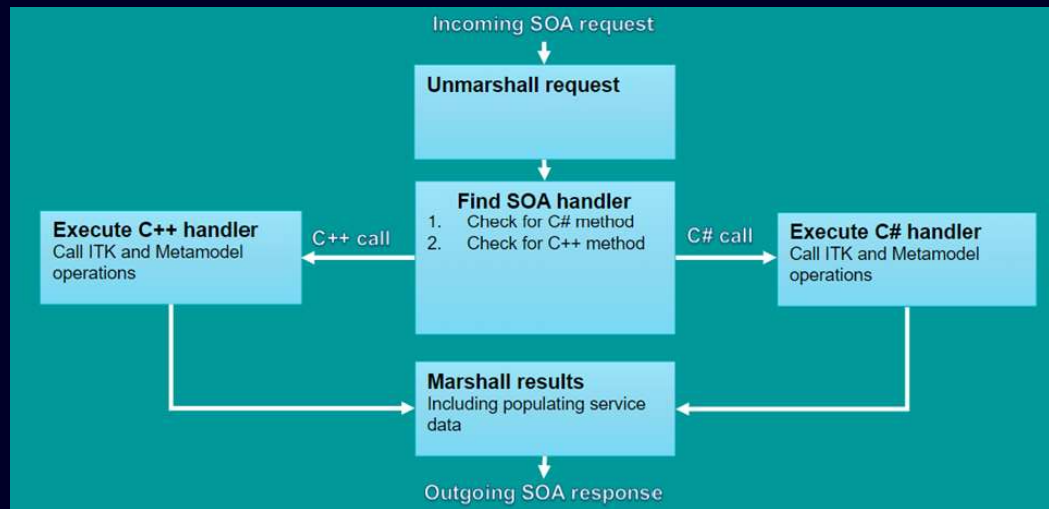
Scripted Business Logic - Pre-requisites



Development environment to write codeful customization of Teamcenter Server in C#

- To develop server-side customization using C#, .NET 8.0 or later SDK is needed
- For C# development, developers would use the supported Integrated Development Environment (IDE) such as Microsoft Visual Studio or Visual Studio Code
- NuGet server is required to host the NuGet packages
- Customizations will work on Windows and Linux platform with Teamcenter installation

Customization of Teamcenter Scripted Business Logic - Scope



The scope of codeful customization of Teamcenter Server in C# includes

- The scope of customization of Teamcenter Server using C# include SOA services, Workflow handler and Metamodel customization
- Metamodel customization includes
 - Business Object operation
 - Runtime properties
 - Operation preconditions, pre-actions and post-actions
 - Property preconditions, pre-actions and post-actions

Customization of Teamcenter

Scripted business logic – Business Use cases and Beta feedback

SOA Services

- Define a SOA service
- Call the SOA service

Operations

- Define a new metamodel operation
- Call an operation with pre/post actions

Runtime Property

- Define a runtime property.
- Get runtime property that returns concatenated values using metamodel wrapper
- Get runtime property that returns concatenated values using ITK wrapper

Extensions

- Define an Extension on metamodel
- Define an Extension on Property
- Post Action on Metamodel Create (fnd0Create) operation

Workflow Handler

- Rule Handler should get invoked
- Action Handler should get invoked

Advanced use case – Extensions

- Post Action on Legacy relation Create (GRM_create) on IMAN_specification relation.
- Post Action on Metamodel Save (fnd0Save) on DocumentRevision

Scripted business logic – Summary

Same flexibility – faster and easier



Efficient, modern development



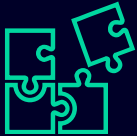
No downtime or compilation



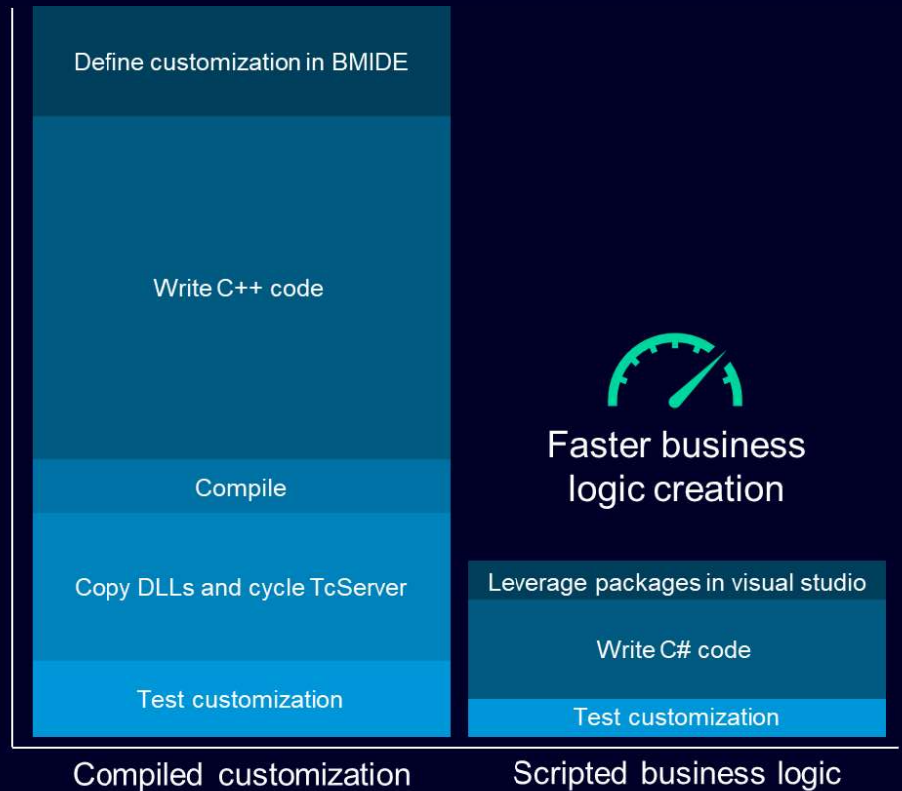
Improved testing with integrated debugging



Works with existing customizations



Teamcenter 2412 Business Logic
Customization Server



Benefits of running the customization a separate server or container

A separate customization server allows teams to develop, test, and deploy custom functionalities for Teamcenter in an isolated, consistent, and scalable environment. A separate customization server enables –

 **Security:** Runs custom logic outside core processes, reducing risk in cloud deployments

 **Scalability:** Easily scale custom Teamcenter services horizontally

 **Portability & Deployment:** Deploy containers quickly across dev, test, and prod environments

 **Isolation for Development:** Self-contained environments prevent system conflicts and streamline customization

 **Performance:** Optimized with a minimal ITK footprint for faster execution

 **Stateless Architecture:** Supports multi-threaded, shared access across multiple TcServers

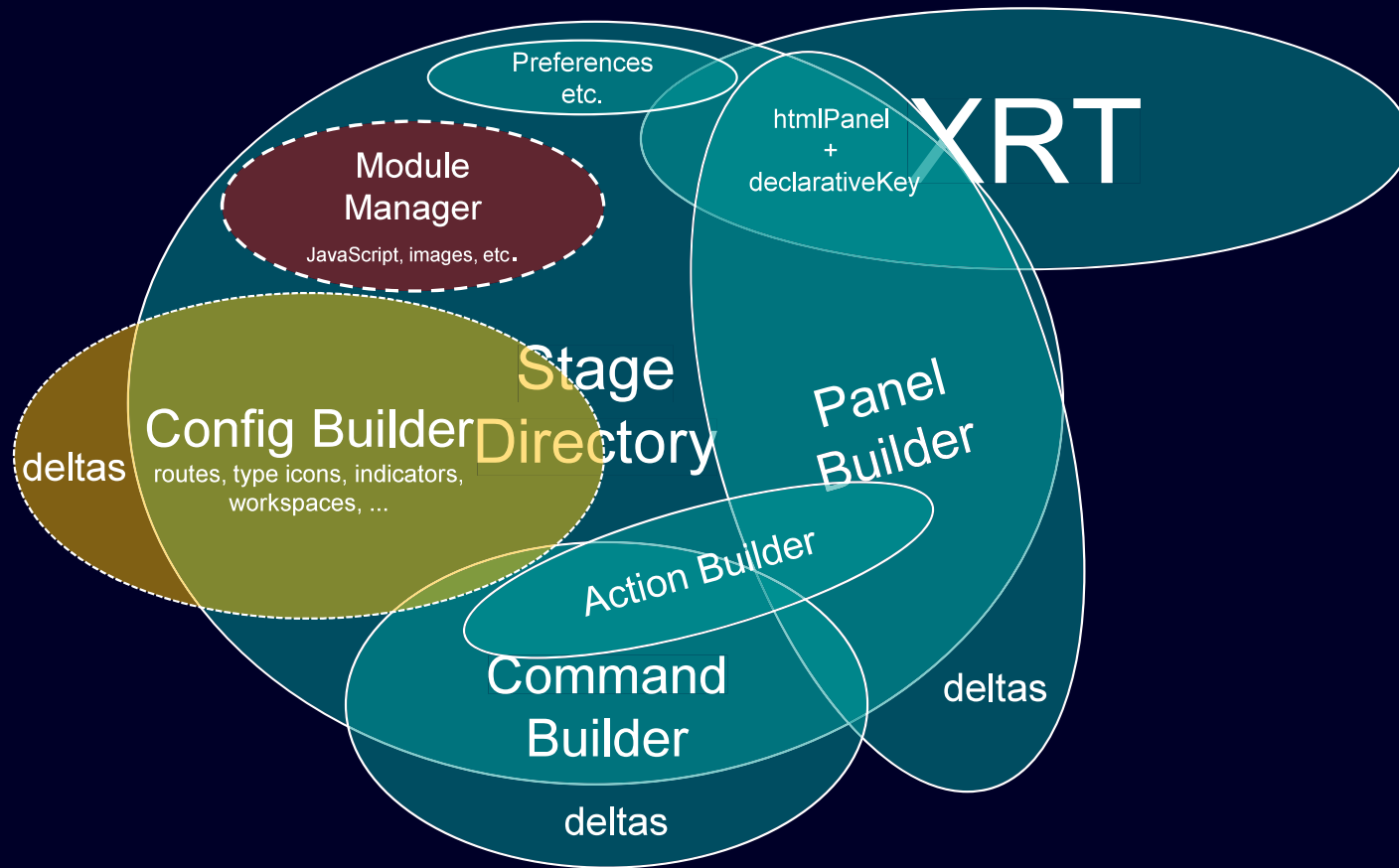
Extend



Customization of Active Workspace

Ways to customize Active Workspace

Ways to Customize Active Workspace



What you can (and are allowed to) customize and how (not including preference-based customizations, table columns, etc.)

Type of customization	Stage Directory	XRT	Command Builder	Panel Builder	Action Builder	Configuration Builder
Create new commands	●		●			
Modify existing commands	●		●			
Create new declarative actions	●				●	
Modify existing declarative actions	●				●	
Create new JavaScript files	●		future	future	future	
Modify existing JavaScript files	●					
Create new declarative components (pages/panels/widgets)	●			●		
Modify existing declarative components (pages/panels/widgets)	●			●		
Create new configuration files	●					2406 (partial)
Modify existing configuration files	●					2406 (partial)
Create and modify XRT tabs		●				
Define declarative tabs	●					future
Create and modify XRT page headers		●				
Define pages	●					2406
Create and modify XRT panels		●				

● Although you technically have physical access to all Active Workspace Client source files via the state/src/repo directory, you are NOT ALLOWED to modify them!

Examples of Client-Side Active Workspace Customizations

The screenshot displays the Siemens XRT Editor interface, which is a client-side active workspace. The interface is divided into three main sections:

- Left Sidebar (Panel Builder):** A vertical sidebar on the left side of the interface, containing a list of folders and files. The folders include "Favorites", "Active Folders", "Home", "Mailbox", "Newstuff", "Test Part", and "Project Data". The "Test Part" folder is currently selected.
- Central Panel (Command Builder):** The main workspace area, which is currently displaying the properties of a selected part, "029354/A;1-Test Part". The panel is divided into two sections: "Part Properties" and "Preview". The "Part Properties" section lists various attributes such as ID, Revision, Name, Description, Type, Release Status, Effectivity, Design Required, Assembly Indicator, Finish Type, Safety Part, Multi-body, Spare Part, Technology Intent, Discontinued, Cancelled, Part Maturity, and Part ECN In. The "Preview" section shows a 3D model of the part.
- Right Panel (XRT Editor):** A panel on the right side of the interface, titled "Information". It contains a "Summary" tab and an "Attachments" tab. The "Summary" tab is currently active, showing a "Preview" section with a 3D model and a "Properties" section with a list of attributes including ID, Revision, Name, Description, Type, Release Status, Date Released, Effectivity, Design Required, Assembly Indicator, Finish Type, Safety Part, Multi-body, Spare Part, and Technology Intent.

Panel Builder

Command Builder

XRT Editor

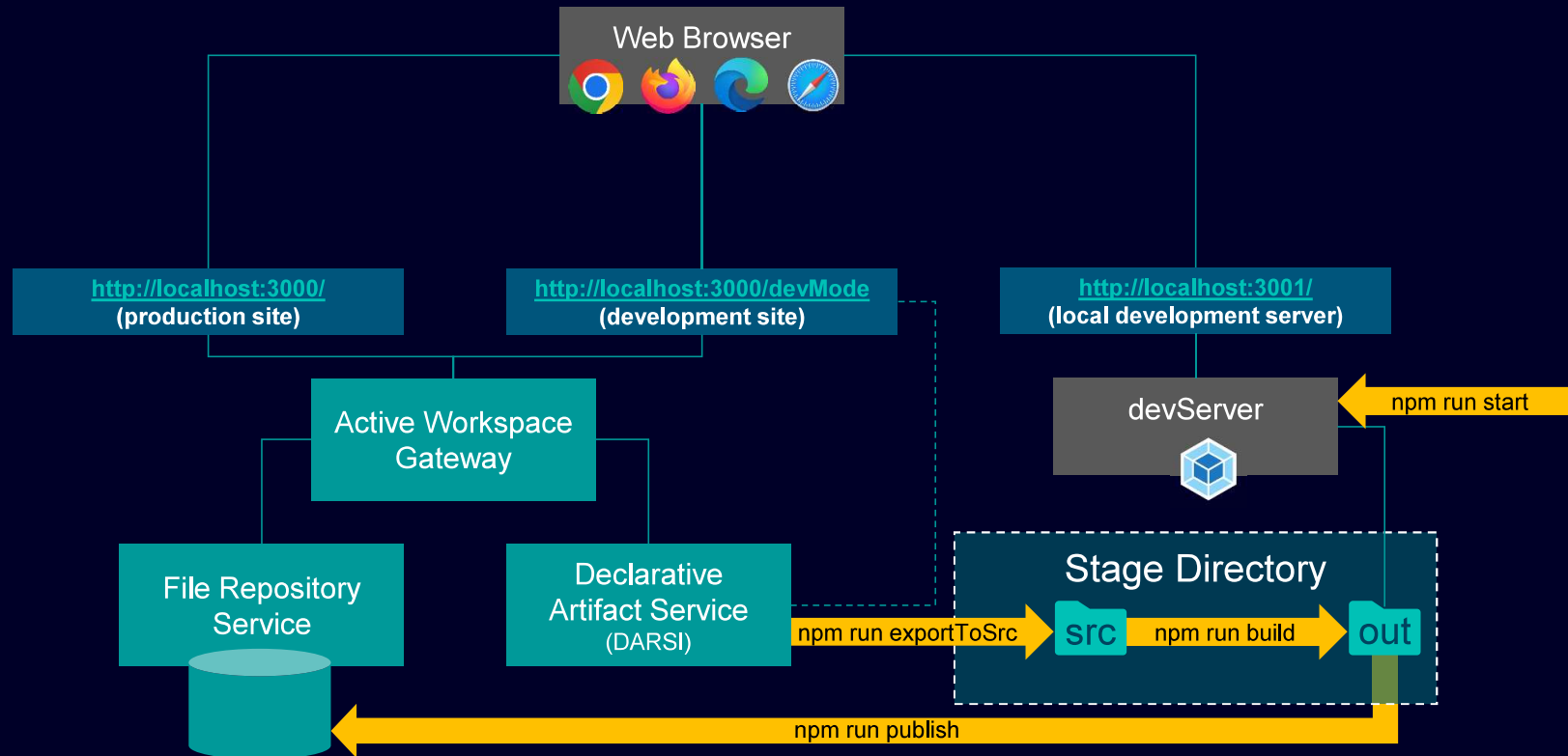
XRT vs Declarative Customizations – Quick Comparison

	XRT	Declarative
Technology	XML	HTML, JSON, JavaScript
Requires rebuilding Active Workspace		●
Requires filesystem access	(only for importing new stylesheets)	● (some use cases may not as of 2412)
Can be used to add/remove properties or datasets on a page	●	●
Supports client-side validation		●
Supports defining complex UI interactions and dynamic behaviors		●
Supports calling SOAs or accessing other web services		●
Supports custom client-side logic		●
Supports <u>safely</u> modifying OOTB UIs	●	● (with UI Builder Editors)
Supports easily overriding UIs for specific users, sites, etc.	●	

NOTE XRT stylesheets can only be used for a *subset* of the Active Workspace UIs (secondary work area content and *some* side panels)

● You should backup your data before modifying OOTB XRT stylesheets!

Building and Publishing Active Workspace Client



Extend

Siemens Web Framework Fundamentals

Configuration points

What is Siemens Web Framework?

Siemens Web Framework (SWF) is a **JavaScript web framework** developed by Siemens DI SW.

Declarative Active Workspace Customizations



Active Workspace



Siemens Web Framework



Third-party JavaScript Framework  React

Declarative Active Workspace customizations are also built using **SWF**, which means that they will also inherit the same benefits.

Active Workspace has been built entirely using **SWF**, to shield it from ever-changing web technologies and enable extensibility.

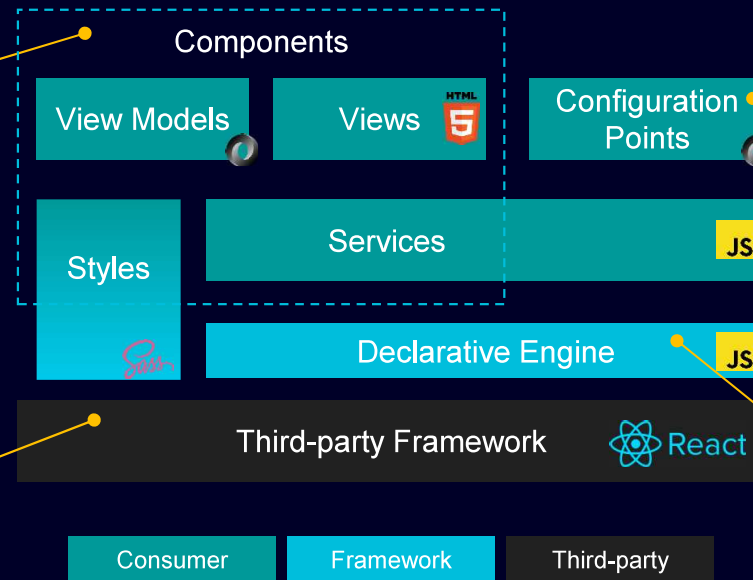
SWF is currently built on top of the popular **React** framework, but in the past (AW v6.1 and prior) it was built on top of **AngularJS**.

SWF

Siemens Web Framework Architecture

Reusable Components

Use or create reusable components in a framework-agnostic way.



Declarative Configuration

Configure key aspects of your application using configuration points, or create your own.

React-based

Although it is never used directly by consumers, SWF is based on React, the most popular JavaScript framework

Abstraction Layer

The declarative engine isolates consumer code from framework-specific code.

Extend

UI Builder

Configuration Over Code

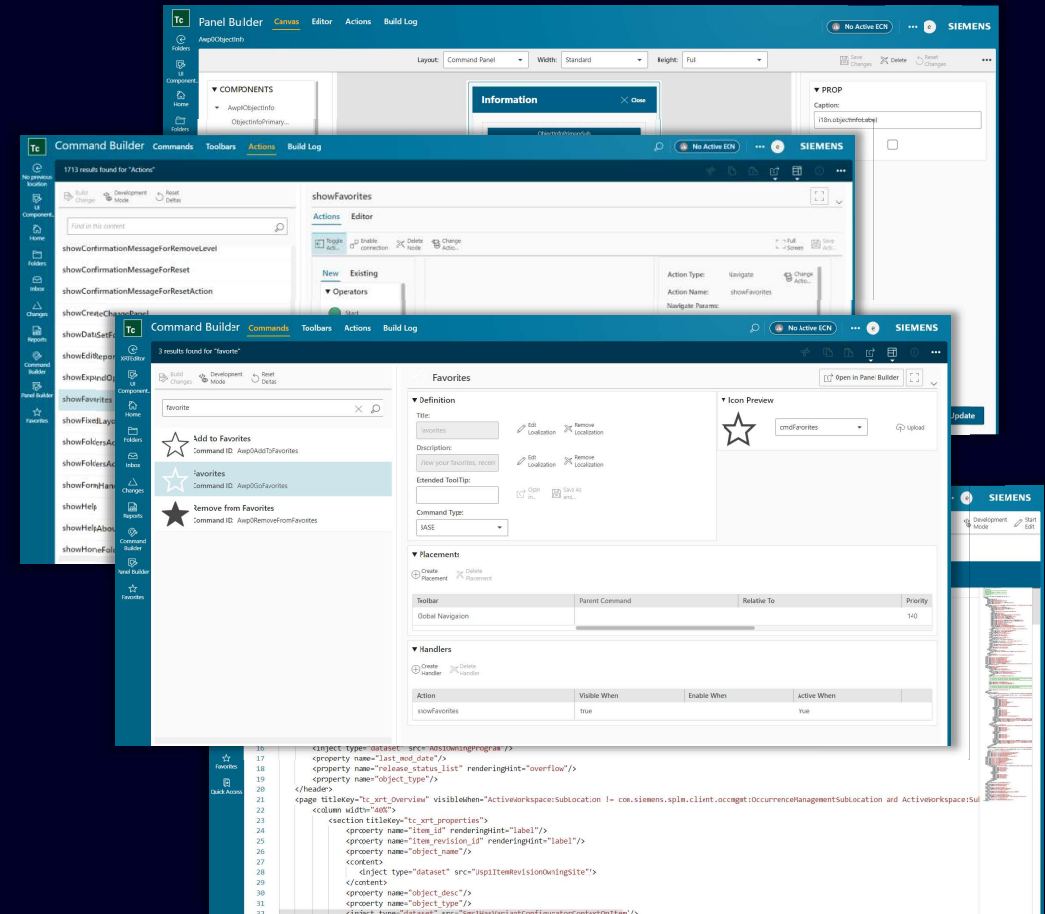
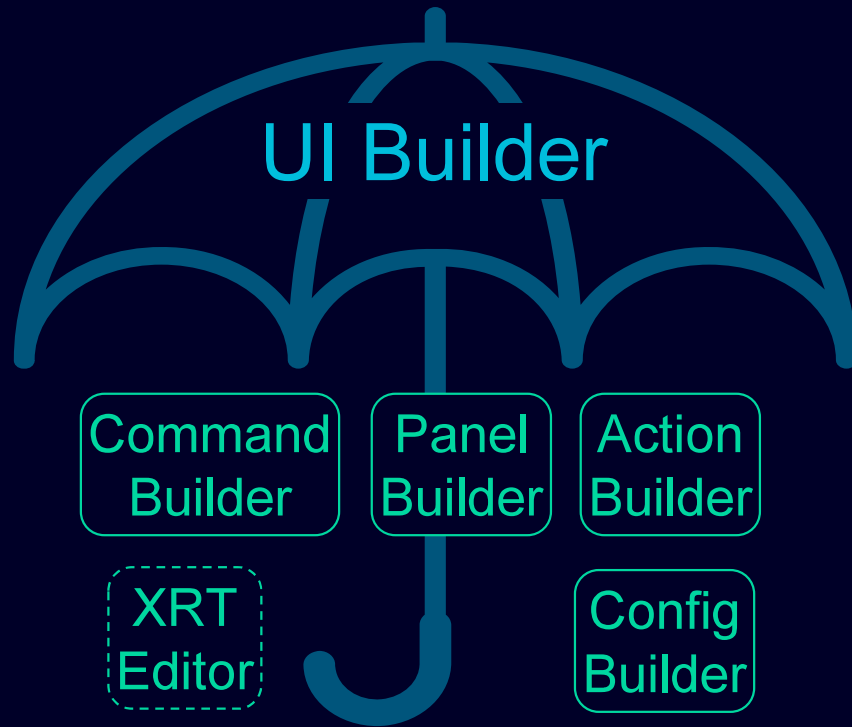
Extend Configuration Over Code



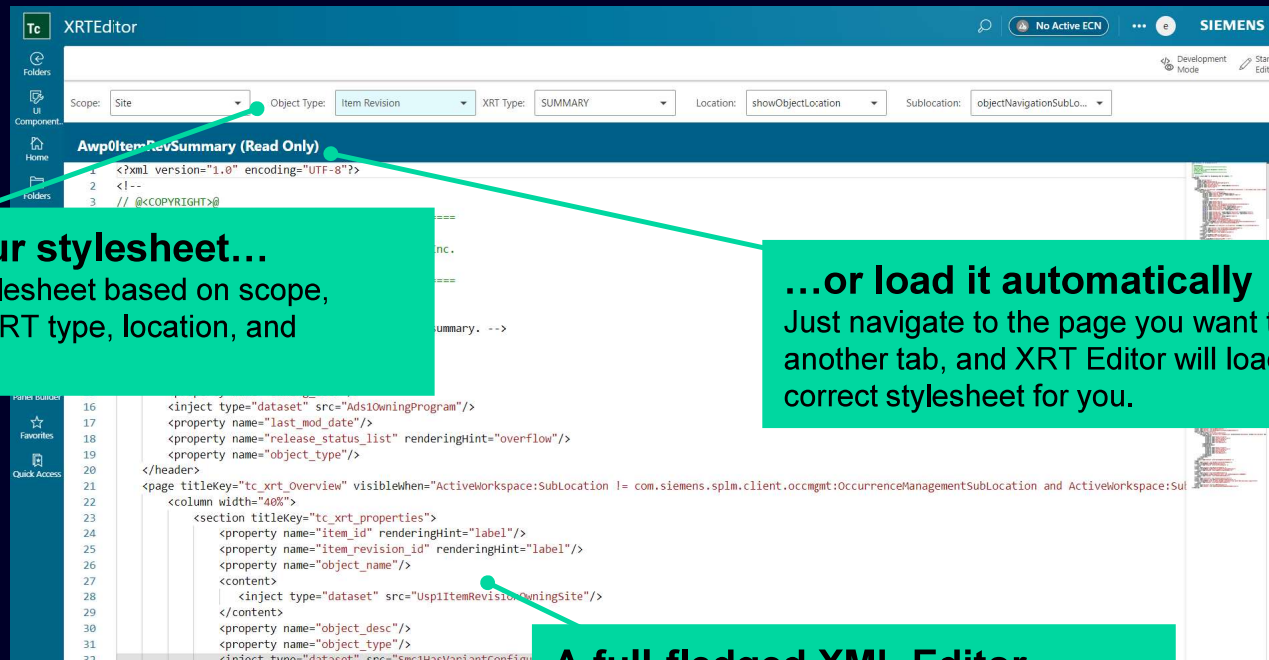
Build new user interface components with drag and drop

- Create and configure new components graphically via drag and drop
- Navigate through the component hierarchy
- Localize your components
- Preview your changes immediately without having to rebuild Active Workspace
- Create and configure actions graphically

What do you mean by “UI Builder”?



XRT Editor



Select your stylesheet...

Load your stylesheet based on scope, object type, XRT type, location, and sublocation.

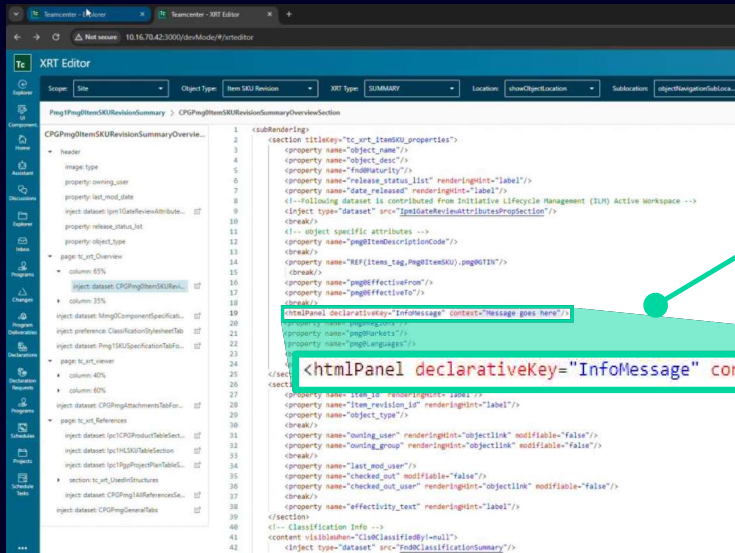
...or load it automatically

Just navigate to the page you want to edit in another tab, and XRT Editor will load the correct stylesheet for you.

A full-fledged XML Editor

Get the full power of Monaco editor right in your browser to edit XRT stylesheets, with syntax highlighting, code folding, and more.

declarativeKey with context



context attribute

You can now specify a **context** attribute to your **htmlPanel** tags. This can be used to pass arbitrary string values to the underlying component.

subPanelContext.declarativeKeyContext

The context attribute can be accessed inside the referenced component via **subPanelContext.declarativeKeyContext**.

```
1 <aw-guidance-message message="infoMessage" show-icon="true" closable="false">
2
3 </aw-guidance-message>
4
```

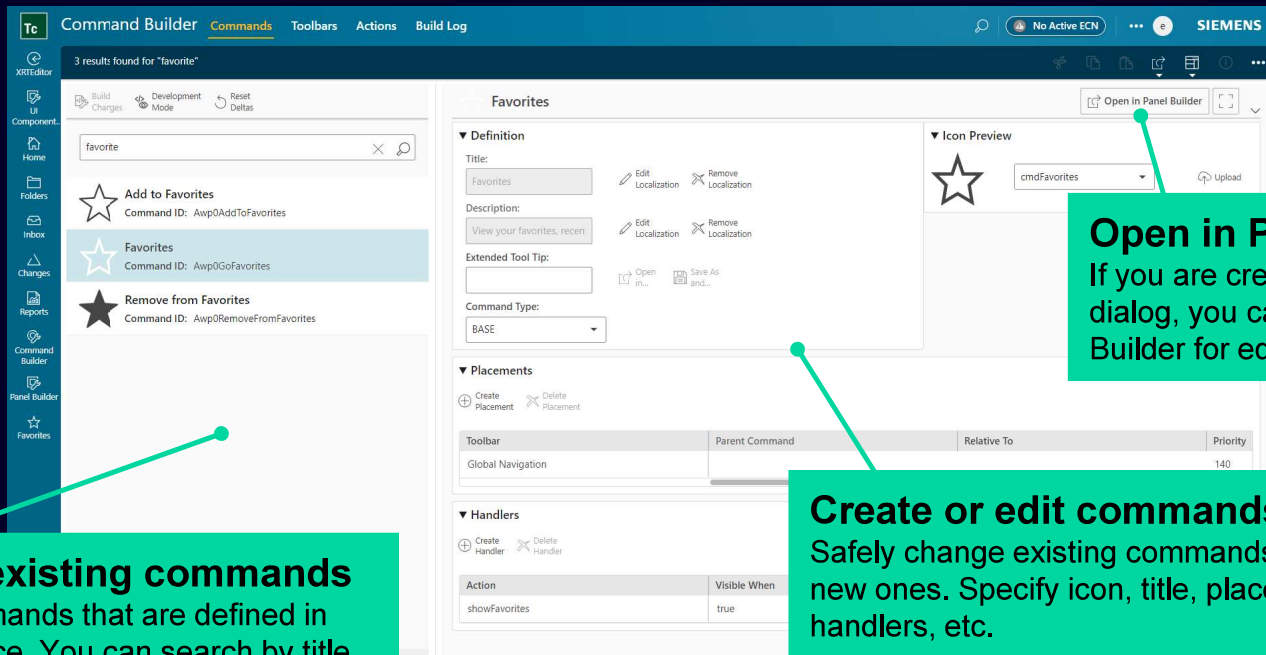
View

```
1 {
2   "schemaVersion": "1.0.0",
3   "imports": [],
4   "data": {},
5   "actions": {},
6   "dataProviders": {},
7   "onEvent": [],
8   "i18n": {},
9   "messages": {
10    "infoMessage": {
11      "messageType": "INFO",
12      "messageText": "{{subPanelContext.declarativeKeyContext}}"
13    }
14  },
15  "conditions": {},
16  "lifecycleHooks": {}
17 }
```

ViewModel



Command Builder



Search for existing commands
Access all commands that are defined in Active Workspace. You can search by title or ID.

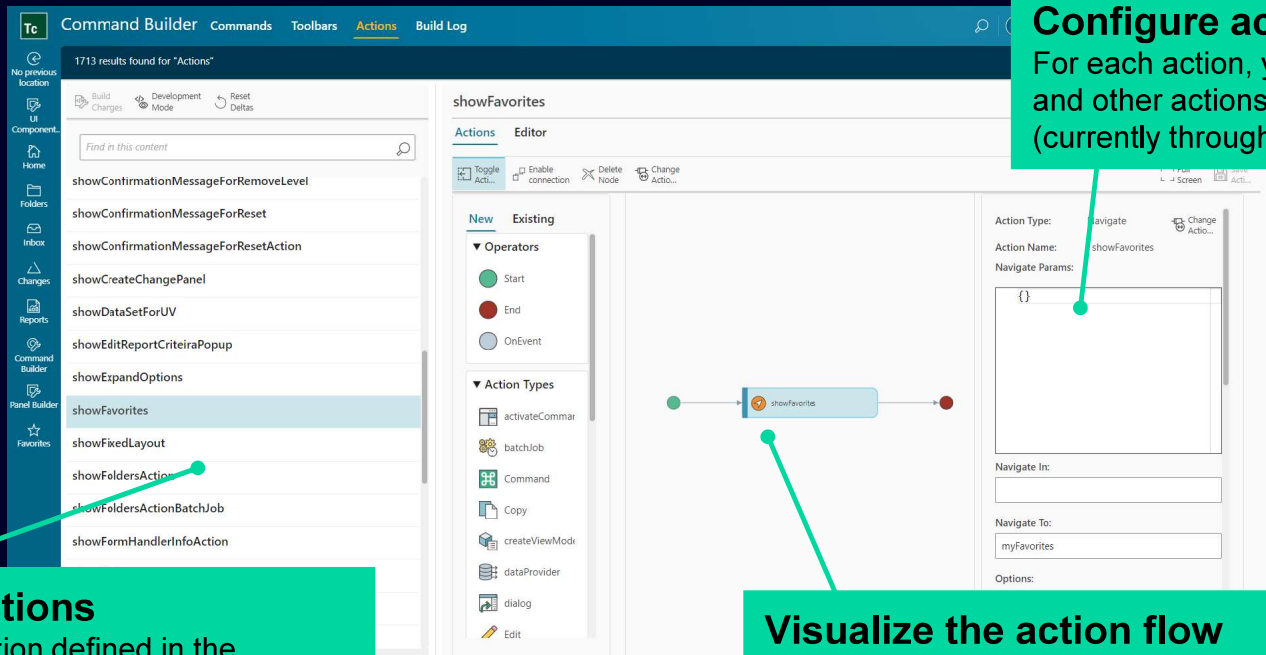
Open in Panel Builder

If you are creating a command that opens a dialog, you can easily open it in Panel Builder for editing.

Create or edit commands

Safely change existing commands or create new ones. Specify icon, title, placement, handlers, etc.

Action Builder



Access all actions

Search for any action defined in the currently-opened view model or used by a command.

Configure action parameters

For each action, you can inspect its flow and other actions that are executed (currently through events only).

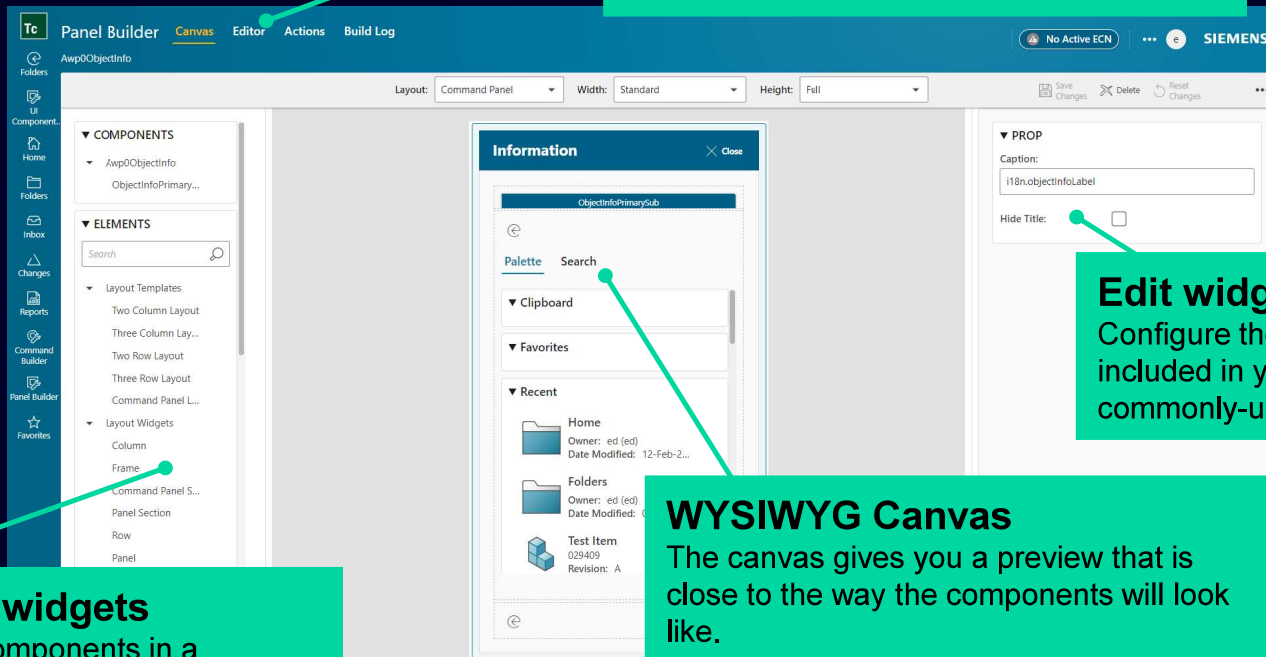
Visualize the action flow

For each action, you can inspect its flow and other actions that are executed (currently through events only).

Panel Builder

Edit your view and view model

Switch to the Editor view to edit the view and view model of your component using Monaco editor.



Drag and drop widgets

Easily create your components in a WYSIWYG way by dragging and dropping common widgets onto a canvas.

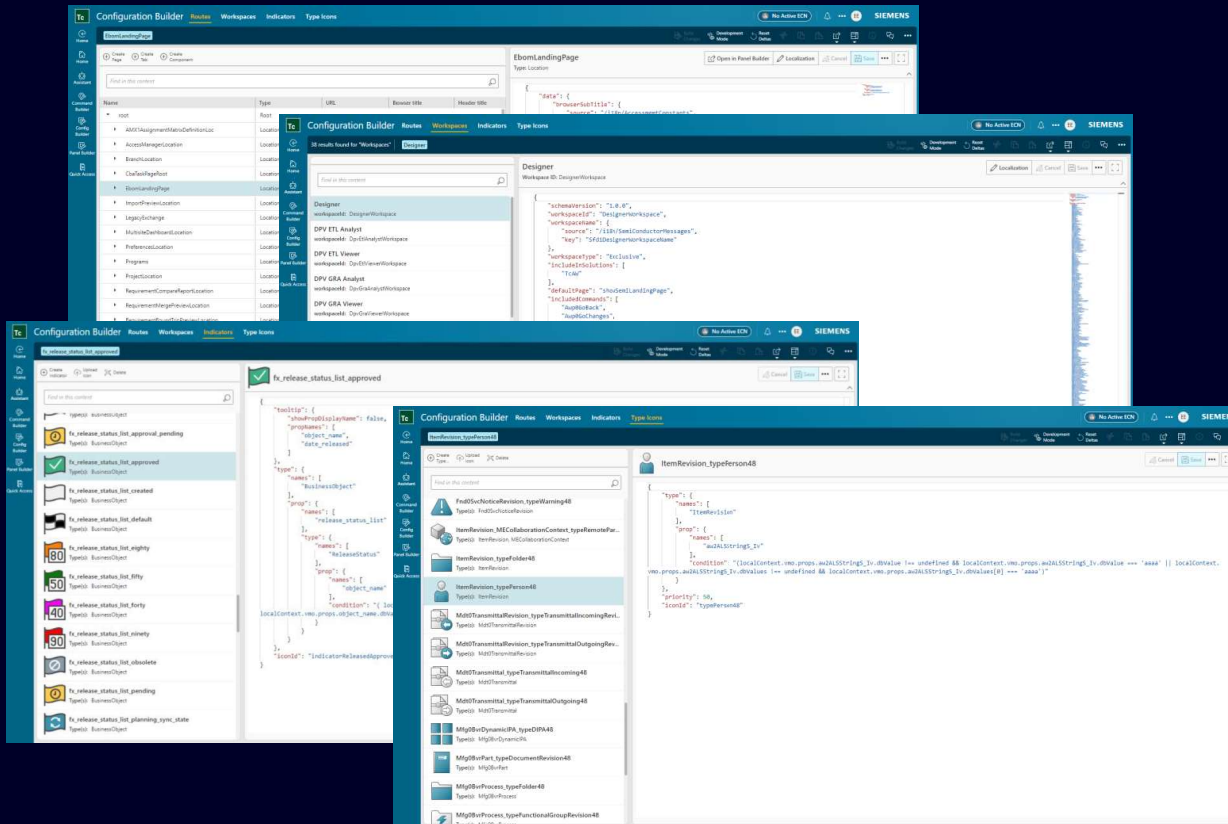
Edit widget properties

Configure the behavior of the widgets included in your components by editing commonly-used properties.

WYSIWYG Canvas

The canvas gives you a preview that is close to the way the components will look like.

Configuration Builder (Teamcenter 2406)



Configuration Builder is a brand new UI Builder editor that can be used to edit common **configuration files**.

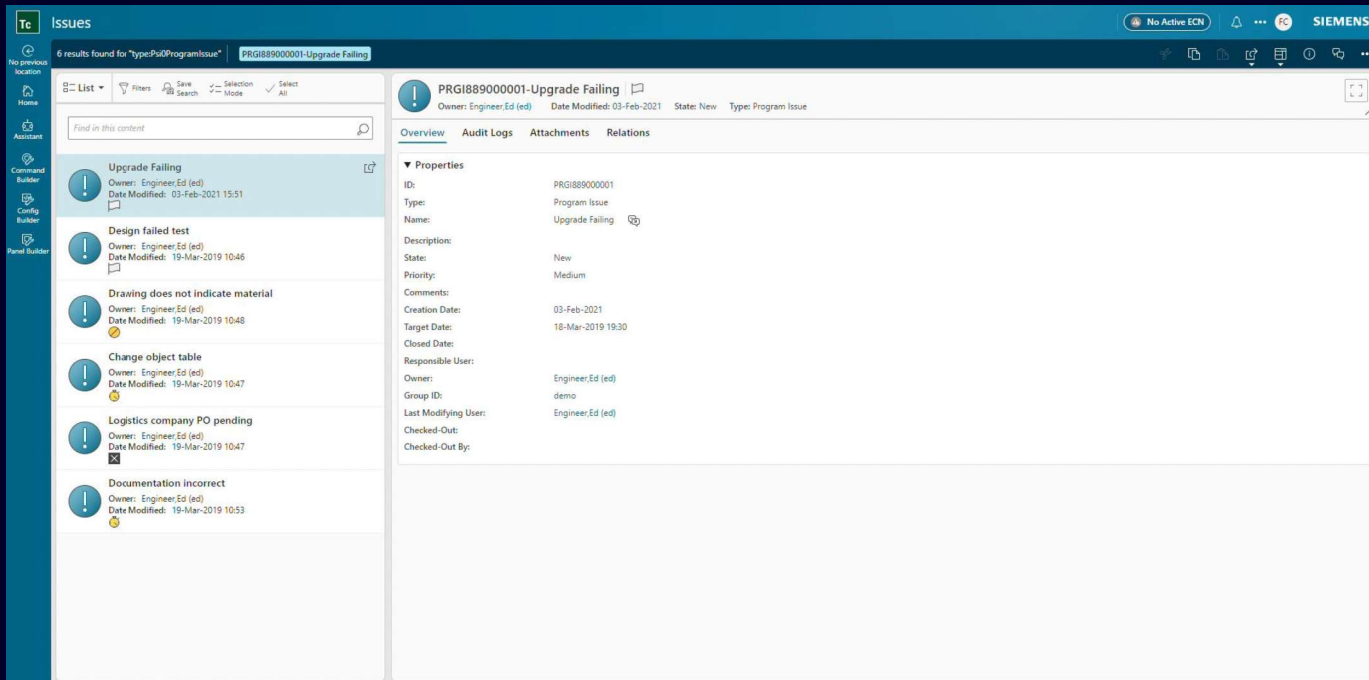
It currently allows you to:

- Create/modify **pages, tabs, components** (states.json)
- Modify existing **workspaces** (workspace*.json)
- Create/modify/delete **indicators** (indicators.json)
- Create/modify/delete **type icons** (typeIconRegistry.json)

Teamcenter 2406

Configuration Builder – Creating a new page

N

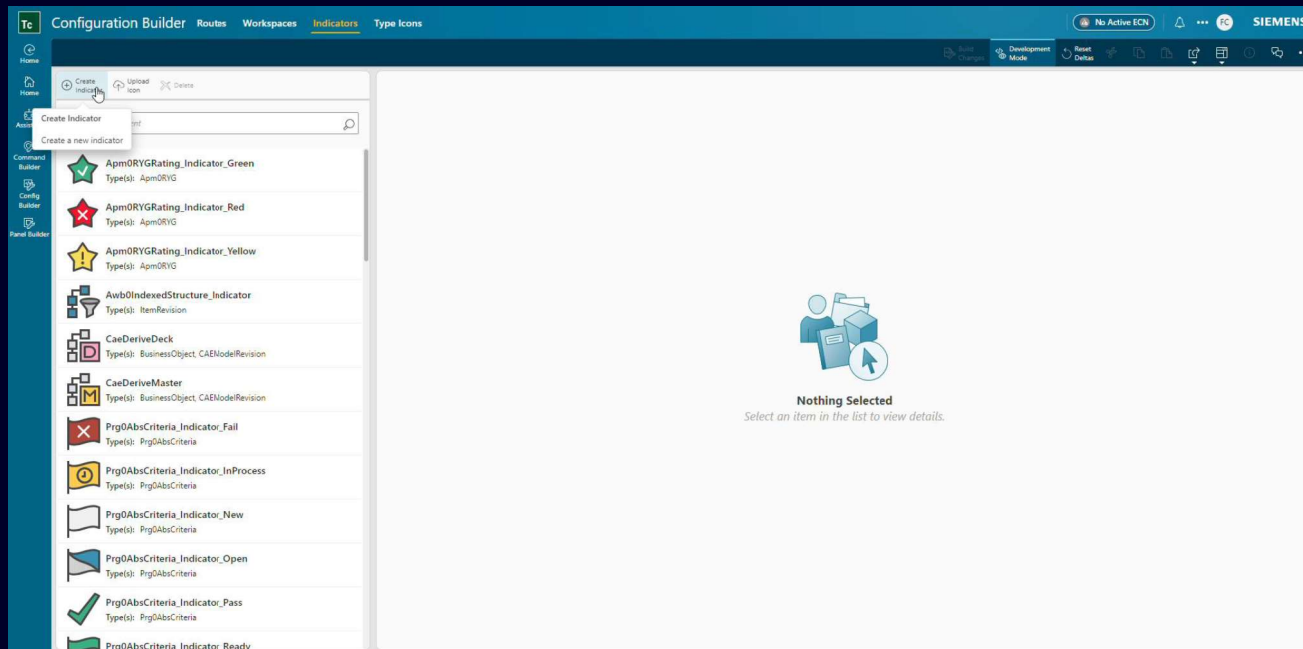


This short demo shows how you can **create a fully-functional search-based location** using Configuration Builder in **two minutes**.

The page will be able to display Program Issues and it will be included in the Active Architect workspace.

Teamcenter 2406

Configuration Builder – Adding a new indicator



This short demo shows how you can **configure a new indicator** for Program Issues of High priority.

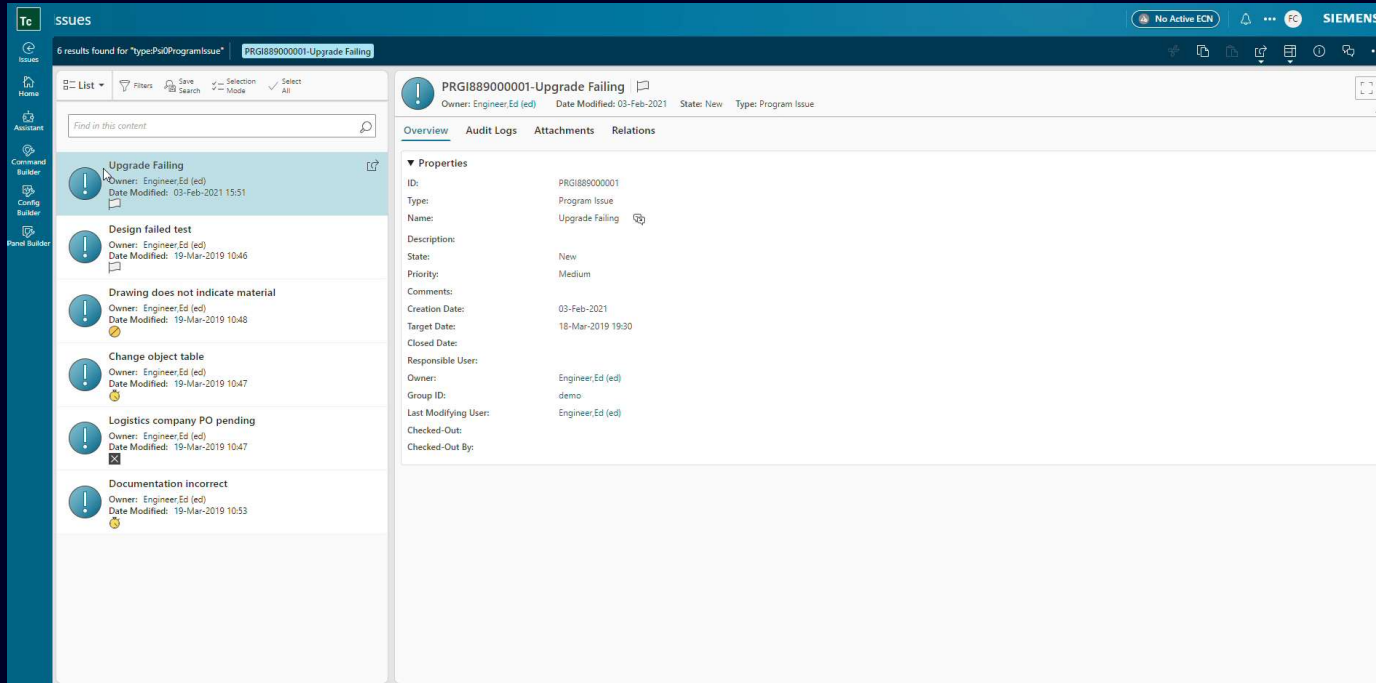
Notice how you can leverage powerful **IntelliSense autocompletion** for indicator icons and for the different properties of the configuration.



Teamcenter 2406

Configuration Builder – Overriding a type icon

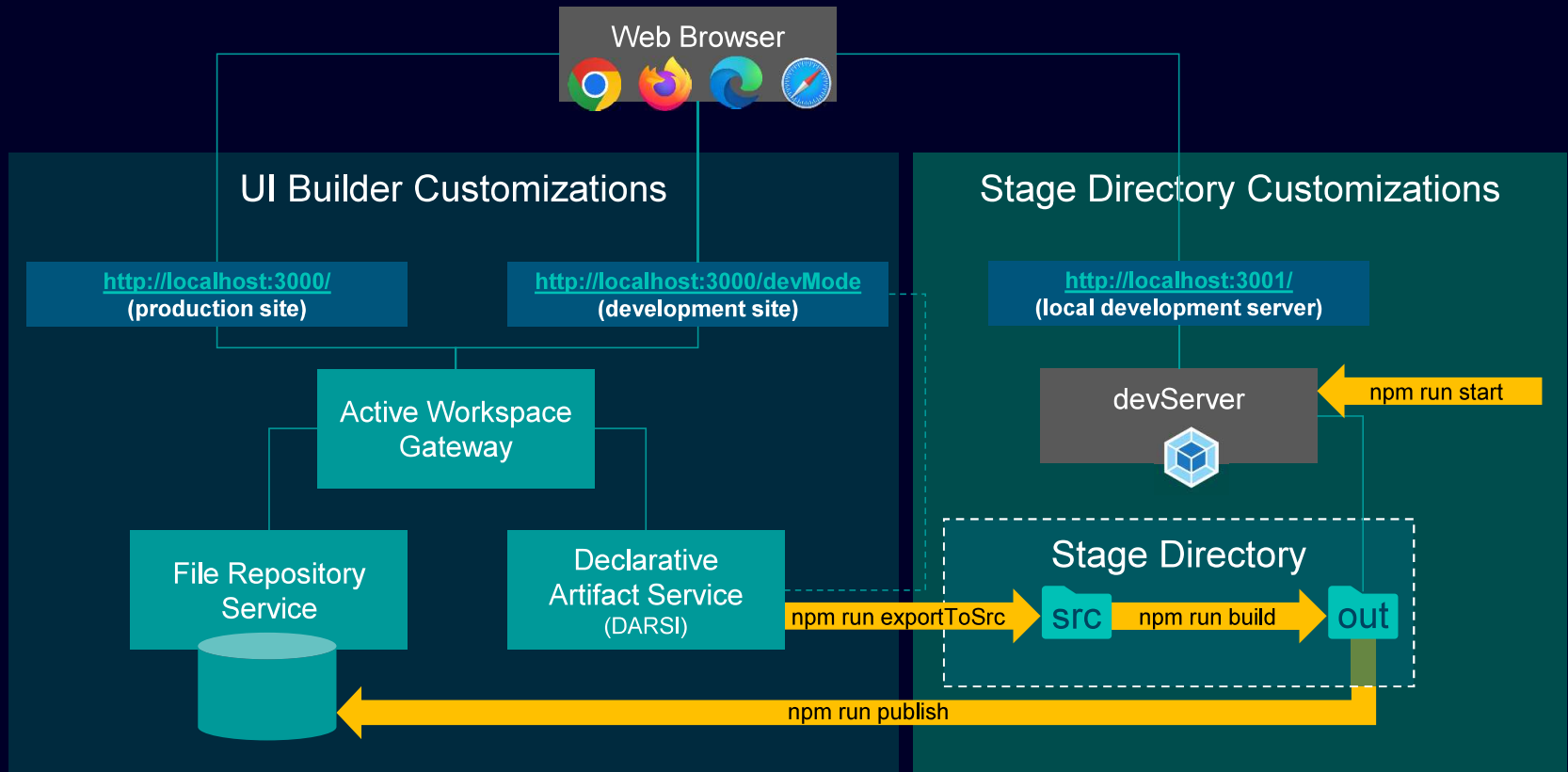
N



This short demo shows how you can **override a type icon** for Program Issues.

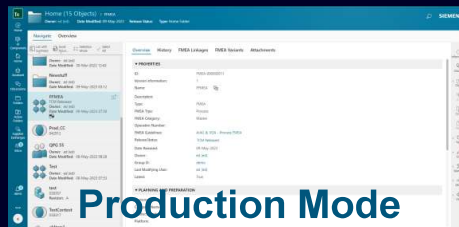
Notice how you can leverage powerful **IntelliSense autocompletion** for type icons and for the different properties of the configuration.

Production & Development Sites vs. Local Development Server

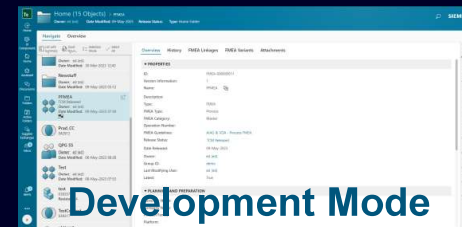


Active Workspace 6.3+ UI Builder – Development Mode vs. Production Mode

<http://localhost:3000/>



<http://localhost:3000/devMode/>



Gateway

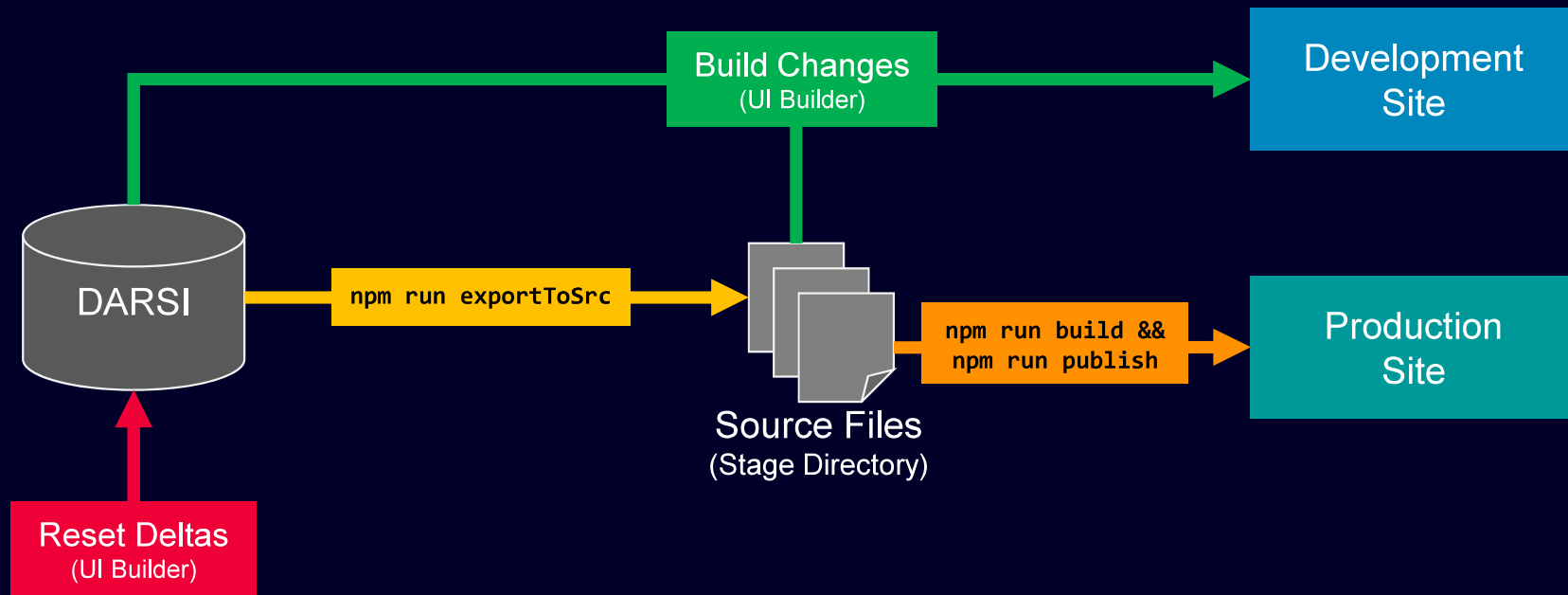
DARSI

UI Definition Files
from Production Build

UI Definition Files
*from Special Preview Build &
UI Builder Overrides (Deltas)*

Active Workspace 6.3+

How to build and publish to the development and production sites



Questions & Answers

Disclaimer

© Siemens 2025

Subject to changes and errors. The information given in this document only contains general descriptions and/or performance features which may not always specifically reflect those described, or which may undergo modification in the course of further development of the products. The requested performance features are binding only when they are expressly agreed upon in the concluded contract.

All product designations may be trademarks or other rights of Siemens AG, its affiliated companies or other companies whose use by third parties for their own purposes could violate the rights of the respective owner.